

Trend Filtering on Graphs: A Self-Contained Explainer for **gflow**

gflow graph trend filtering project notes

Build 2026-05-16 10:10:45 EDT

Abstract

This note explains the main ideas behind one-dimensional trend filtering, graph trend filtering, graph fused lasso, and related recent work in a way intended to guide a future **gflow** implementation. The emphasis is on accessible mathematics: each formula is introduced by the modeling question it answers, then interpreted in terms of what the estimator does to a signal. The central theme is the contrast between quadratic graph smoothing, which spreads smoothness globally, and ℓ_1 -adaptive graph smoothing, which can preserve localized changes by making graph differences sparse.

1 Purpose and Scope

The immediate motivation for this document is practical. In the path-graph benchmark for `fit.metric.graph.lowpass()`, the strongest classical one-dimensional baseline was a wrapper around `genlasso::trendfilter()`. That result suggests that a graph analogue of trend filtering may be an important comparator to add to **gflow**.

This document therefore has four goals:

1. explain one-dimensional trend filtering from first principles;
2. show how ordered finite differences become graph incidence and Laplacian operators;
3. summarize the main published ideas on trend filtering and fused lasso over general graphs;
4. identify the implementation decisions that matter for **gflow**, especially for weighted metric graphs.

The document is not a full literature review. It is a design-oriented mathematical explainer.

2 Why Trend Filtering?

Suppose we observe noisy values y_i of an unknown signal. A smoother has to decide how much of the variation in y_i is real signal and how much is noise. Standard quadratic smoothers penalize squared roughness. This is powerful, but it tends to smooth everywhere. Trend filtering uses a different principle: penalize the absolute values of discrete derivatives.

quadratic smoothing: many small penalties everywhere

ℓ_1 trend filtering: many exact zeros, a few active bends

That exact-zero behavior is the lasso mechanism. In ordinary lasso, an ℓ_1 penalty sets many regression coefficients to zero. In trend filtering, an ℓ_1 penalty sets many discrete derivative values

to zero. The result is a curve that is simple over long stretches but can bend locally where the data demand it.

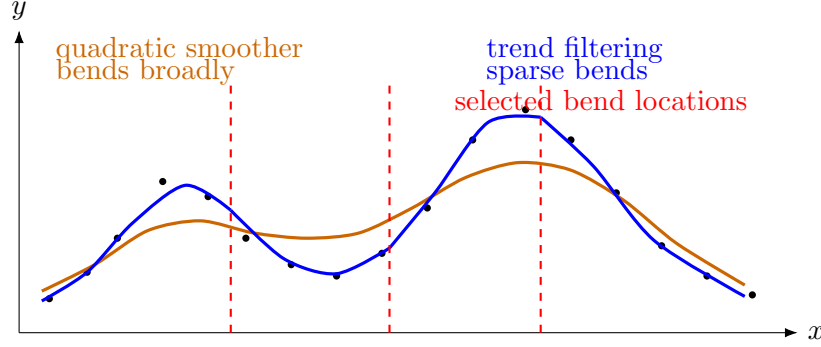


Figure 1: A conceptual comparison. A quadratic smoother spreads curvature continuously; trend filtering keeps many higher-order differences exactly zero and concentrates bends at selected locations.

3 One-Dimensional Trend Filtering

3.1 Data and Fitted Values

Start with ordered observations

$$x_1 < x_2 < \cdots < x_n, \quad y_i = f(x_i) + \varepsilon_i.$$

Trend filtering estimates fitted values

$$\theta = (\theta_1, \dots, \theta_n)^T, \quad \theta_i \approx f(x_i).$$

For now, assume the x_i are equally spaced. Unequal spacing can be handled by divided-difference versions of the same operators.

3.2 Finite Differences

The first difference measures local change:

$$(D^{(1)}\theta)_i = \theta_{i+1} - \theta_i.$$

The second difference measures local curvature:

$$(D^{(2)}\theta)_i = \theta_{i+2} - 2\theta_{i+1} + \theta_i.$$

The third difference measures change in curvature:

$$(D^{(3)}\theta)_i = \theta_{i+3} - 3\theta_{i+2} + 3\theta_{i+1} - \theta_i.$$

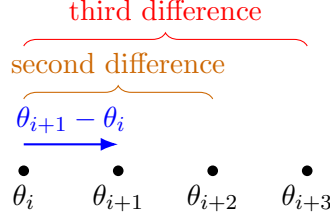


Figure 2: Trend filtering penalizes finite differences. Higher order differences measure higher order changes in the fitted curve.

3.3 The Optimization Problem

Order- k trend filtering solves

$$\hat{\theta}_\lambda = \arg \min_{\theta \in \mathbb{R}^n} \left\{ \frac{1}{2} \|y - \theta\|_2^2 + \lambda \left\| D^{(k+1)} \theta \right\|_1 \right\}.$$

The first term is data fidelity. The second term is an ℓ_1 penalty on the $(k+1)$ st discrete derivative.

Order k	Penalized quantity	Shape of fitted signal
0	first differences	piecewise constant
1	second differences	piecewise linear
2	third differences	piecewise quadratic

The path-graph benchmark used `genlasso::trendfilter(y, pos = x, ord = 2)`, so it used $k = 2$. That means it estimated a locally piecewise quadratic curve and selected the amount of smoothing by cross-validation.

4 From a Line to a Graph

The path graph is the bridge between one-dimensional trend filtering and graph trend filtering. On a line, each point has a left and right neighbor. On a graph, each vertex has graph neighbors. The replacement is:

$$\text{ordered finite differences} \longrightarrow \text{edge differences}.$$

Choose an arbitrary orientation for every undirected edge. If edge e is oriented from vertex i to vertex j , define one row of the incidence matrix D by

$$D_{e,i} = -1, \quad D_{e,j} = 1, \quad D_{e,\ell} = 0 \quad (\ell \notin \{i, j\}).$$

Then

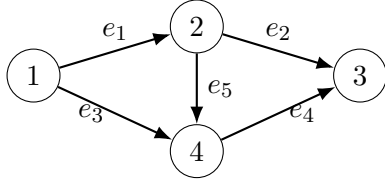
$$(D\beta)_e = \beta_j - \beta_i.$$

The graph Laplacian is

$$L = D^T D.$$

For an unweighted graph, $L\beta$ compares each vertex value to the values of its neighbors:

$$(L\beta)_i = \sum_{j \sim i} (\beta_i - \beta_j).$$



$D\beta$ has one entry per edge.

If $e_1 : 1 \rightarrow 2$, then

$$(D\beta)_{e_1} = \beta_2 - \beta_1.$$

The chosen orientation only sets signs;

ℓ_1 penalties use absolute values.

Figure 3: The incidence matrix turns a vertex signal into edge differences.

5 Graph Trend Filtering

Wang, Sharpnack, Smola, and Tibshirani define graph trend filtering by constructing higher-order graph difference operators from D and L . The estimator is

$$\hat{\beta}_\lambda = \arg \min_{\beta \in \mathbb{R}^n} \left\{ \frac{1}{2} \|y - \beta\|_2^2 + \lambda \left\| \Delta^{(k+1)} \beta \right\|_1 \right\}.$$

One convenient expression for their graph difference operators is

$$\Delta^{(k+1)} = \begin{cases} L^{(k+1)/2}, & k \text{ odd}, \\ DL^{k/2}, & k \text{ even}. \end{cases}$$

Order k	Operator	Penalized object	Graph interpretation
0	D	edge differences	piecewise constant regions
1	L	vertex Laplacian values	graph piecewise linear analogue
2	DL	edge differences of Laplacian values	graph piecewise quadratic analogue

5.1 $k = 0$: Graph Fused Lasso

For $k = 0$, $\Delta^{(1)} = D$, so

$$\left\| \Delta^{(1)} \beta \right\|_1 = \sum_{(i,j) \in E} |\beta_i - \beta_j|.$$

The estimator is

$$\hat{\beta} = \arg \min_{\beta \in \mathbb{R}^n} \left\{ \frac{1}{2} \|y - \beta\|_2^2 + \lambda \sum_{(i,j) \in E} |\beta_i - \beta_j| \right\}.$$

This is graph fused lasso or graph total variation denoising.

5.2 $k = 1$: Penalizing $L\beta$

For $k = 1$, the operator is

$$\Delta^{(2)} = L.$$

The penalty is

$$\|L\beta\|_1 = \sum_i \left| \sum_{j \sim i} (\beta_i - \beta_j) \right|.$$

On a path graph, $L\beta$ is essentially a second difference. On a general graph, it measures whether a vertex value agrees with a local average of its neighbors. Sparsity in $L\beta$ means many vertices behave locally like graph-harmonic points.

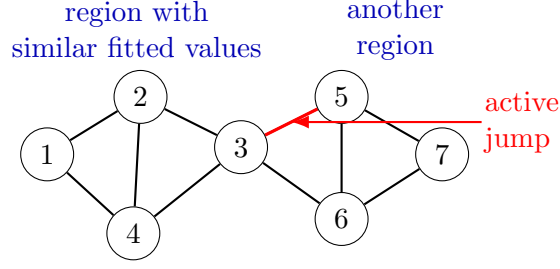


Figure 4: The $k = 0$ penalty encourages piecewise constant graph regions while allowing selected edges to carry jumps.

5.3 $k = 2$: Penalizing $DL\beta$

For $k = 2$, the operator is

$$\Delta^{(3)} = DL.$$

This takes the graph Laplacian values $L\beta$ at vertices and then computes edge differences between those values. On a path graph this corresponds to a third-difference penalty, which is why the analogy is piecewise quadratic.

6 Weighted Graphs and Metric Conductances

The published graph trend filtering construction is often introduced using an unweighted incidence matrix. In `gflow`, graphs usually carry metric edge lengths or conductances, so weighting is not a detail. It is a modeling choice.

For edge $e = (i, j)$, a weighted incidence operator can be written as

$$(D_w\beta)_e = \omega_e(\beta_j - \beta_i).$$

Then the graph fused lasso penalty becomes

$$\|D_w\beta\|_1 = \sum_{e=(i,j)} \omega_e |\beta_j - \beta_i|.$$

If c_e is a conductance, two natural conventions are:

$$\omega_e = c_e \quad \text{or} \quad \omega_e = \sqrt{c_e}.$$

The first convention makes the ℓ_1 edge penalty scale directly with conductance:

$$\sum_e c_e |\beta_j - \beta_i|.$$

The second convention aligns with the quadratic Laplacian identity:

$$\|D_w\beta\|_2^2 = \beta^T L_w \beta, \quad L_w = D_w^T D_w.$$

This choice should probably be explicit in the API. A first implementation could expose a weighting option such as

$$\omega_e \in \{1, c_e, \sqrt{c_e}\}.$$

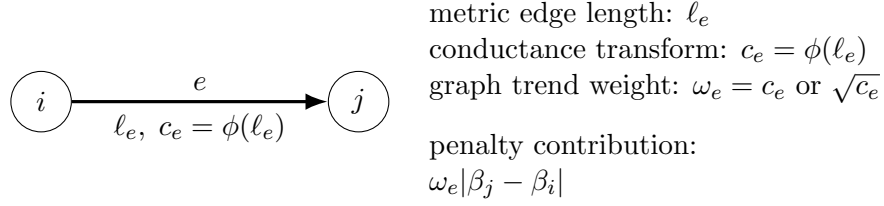


Figure 5: For metric graph trend filtering, `gflow` must decide how metric lengths become ℓ_1 edge-difference weights.

7 Relationship to Metric Graph Low-Pass Smoothing

The metric graph low-pass smoother is fundamentally ℓ_2 -spectral. A closely related variational form is

$$\min_f \left\{ \sum_i (y_i - f_i)^2 + \eta f^T L f \right\}.$$

The graph trend filtering analogue is

$$\min_{\beta} \left\{ \sum_i (y_i - \beta_i)^2 + \lambda \left\| \Delta^{(k+1)} \beta \right\|_1 \right\}.$$

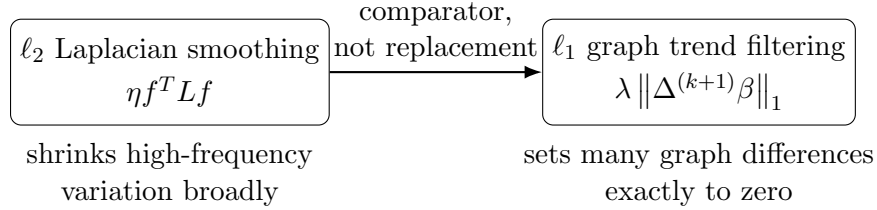


Figure 6: Graph trend filtering is the ℓ_1 -adaptive counterpart to ℓ_2 graph low-pass smoothing.

8 Published Ideas to Build On

8.1 Trend Filtering on Graphs

Yu-Xiang Wang, James Sharpnack, Alex Smola, and Ryan J. Tibshirani introduced the graph trend filtering estimator and studied its statistical behavior.

- arXiv: <https://arxiv.org/abs/1410.7690>
- JMLR PDF: <https://jmlr.csail.mit.edu/papers/volume17/15-147/15-147.pdf>
- Main contribution: graph difference operators $\Delta^{(k+1)}$ and locally adaptive graph denoising.

8.2 The DFS Fused Lasso

Madrid Padilla, Sharpnack, Scott, and Tibshirani focused on scalable graph fused lasso, corresponding mainly to the $k = 0$ case. The idea is to use a depth-first-search traversal to reduce a graph denoising problem to a chain-like problem while retaining useful guarantees.

- JMLR page: <https://www.jmlr.org/beta/papers/v18/16-532.html>
- Main contribution: fast graph total variation denoising over general graphs.

8.3 Fused ℓ_1 Trend Filtering on Graphs

Vladimir Pastukhov’s recent arXiv work studies fused ℓ_1 trend filtering on graphs, including combinations of trend filtering and fusion regularizers.

- arXiv: <https://arxiv.org/abs/2401.05250>
- Main contribution: broader fused graph trend filtering objectives and scalable iterative schemes.

9 Implementation Implications for gflow

A conservative implementation path is:

1. Start with $k = 0$, the weighted graph fused lasso:

$$\hat{\beta} = \arg \min_{\beta \in \mathbb{R}^n} \left\{ \frac{1}{2} \|y - \beta\|_2^2 + \lambda \|D_w \beta\|_1 \right\}.$$

2. Add an operator-construction helper that returns the edge table, orientation, weights, incidence triplets, and diagnostics.
3. Make the metric length to trend weight convention explicit.
4. Add path-graph checks against one-dimensional fused-lasso or trend filtering references.
5. Add higher-order $k = 1$ and $k = 2$ operators once the solver and weighting conventions are stable.

10 Open Design Questions

1. Should edge weights in D_w be c_e , $\sqrt{c_e}$, or another transform of metric length?
2. Should the first solver be exact for small/medium graphs, approximate for large graphs, or both?
3. Should disconnected graphs be fit componentwise?
4. Should the API expose both fixed- λ refits and cross-validated λ selection?
5. Should DFS fused lasso be an approximation mode or a separate estimator?

11 Summary

One-dimensional trend filtering works by penalizing discrete derivatives with an ℓ_1 norm. Graph trend filtering replaces ordered finite differences with graph incidence and Laplacian-based difference operators. The resulting estimators are locally adaptive graph smoothers: they can be simple over large regions while preserving localized changes. That makes them natural comparators to gflow’s metric graph low-pass smoother and a promising next project for graph-based regression experiments.